DIM0501 — Boas Práticas de Programação

Relatório do Projeto da Unidade 1 — AgendaFácil

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	Nome Sobrenome (matrícula)
Nome Sobrenome (matrícula)	
Linguagem	Python
Repositório ou arquivo entregue	`https://github.com/usuario/repositorio`

1. Diagnóstico do código-base

#	Categoria	Anomalia e onde está	Por que é um problema
1	Programação defensiva e segurança	Em `add`, entradas fora das regras são ignoradas em silêncio: os `if` aninhados de `h` e `dur` não têm `else`, e todo o corpo está dentro de um `try` com `except: pass`. Ex.: marcar às 20h não marca e não avisa.	Quem utiliza a função acredita que o atendimento foi marcado, pois nenhuma mensagem ou exceção é apresentada quando uma regra não é atendida. Isso faz com que o problema só seja percebido posteriormente, quando o paciente descobre que não possui horário, sem que exista qualquer indicação da causa. Além disso, o `except: pass` faz com que qualquer erro que venha a acontecer dentro de `add`, inclusive bugs do próprio código, seja ignorado, dificultando a identificação de onde um determinado problema pode estar acontecendo.
2	Manutenabilidade	Em `add`, os parâmetros da função não possuem nomes descritivos.	Para entender o que é cada um desses parâmetros, o desenvolvedor precisará navegar pelo código e tentar inferir com base nas verificações o que cada parâmetro significa, além de ter que ter algum domínio pequeno sobre o negócio para facilitar essa inferência.
3	Manutenabilidade	Em todas as funções presentes no sistema, ao ter que acessar alguma informação de um agendamento existente, é feita a obtenção desses dados com base na posição em que elas são armazenadas na estrutura de dados utilizada (lista).	Torna-se necessário o conhecimento da ordem aos quais os dados são armazenados para cada uma das linhas. Caso utilizassemos uma estrutura diferente como um objeto ou mesmo um dicionário, onde temos a associação entre chave e valor, onde essa chave indicaria exatamente qual informação que estariamos acessando (nome do paciente, data e etc.) e o valor sendo exatamente a informação ao qual armazenamos durante a inserção, isso tornaria muito mais fácil o processo de decodificar qual informação está sendo acessa, ou mesmo durante o processo de implementar uma funcionalidade diferente, acessar mais facilmente um valor ou alterar ele se torna muito mais simples por retirar a necessidade de ficar sempre se lembrando da ordem dos dados.
4	Adequação e lógica	Em `add`, não há nenhuma validação referente ao formato de data.	De acordo com a regra de negócio, a data deverá assumir o seguinte formato: AAAA-MM-DD. Qualquer outro formato é considerado como inválido.
5	Adequação e lógica	Em `add`, a duração não contém uma validação adequada, conforme defi-	De acordo com a regra de negócio, a duração deve ser entre 15 e 120 mi-

2. O que foi resolvido e o que ficou como dívida

#	Situação	Como foi resolvida, ou por que ficou
1	Resolvida	O `try/except: pass` foi removido. Cada regra violada lança uma exceção específica (horário, duração, data, conflito e agendamento inexistente), e erros inesperados sobem para quem chamou.
2	Resolvida	Os parâmetros passaram a se chamar `nome_paciente`, `nome_doutor`, `data`, `hora_atendimento` e `duracao_minutos`, e as funções `add`, `rem` e `lst` viraram `adicionar`, `remarcar` e `listar_por_data`, seguindo a PEP 8.
3	Resolvida	O atendimento virou a classe `Agendamento`, com campos nomeados. O status deixou de ser a string solta `"MARCADO"` e passou a usar `AgendamentoStatus`.
4	Resolvida	`validar_data` confere o formato AAAA-MM-DD em `adicionar` (pelo `Agendamento`), `remarcar` e `listar_por_data`.
5	Resolvida	`validar_duracao` exige de 15 a 120 minutos, em múltiplos de 15. Os limites viraram constantes nomeadas, e cada regra tem sua própria exceção.
6	Dívida assumida	A busca continua percorrendo a lista. No volume de uma clínica (dezenas de atendimentos por dia) o custo é desprezível, e indexar por data, hora e profissional exigiria manter índices sincronizados a cada remarcação. Como a busca está centralizada em `__buscar_agendamento`, a troca, se for necessária, fica em um lugar só.
7	Resolvida	As regras de hora, duração e data foram para módulos próprios (`agenda_utilitario` e `data_utilitarios`), o conflito é verificado só em `__validar_disponibilidade_horario` (usado por `adicionar` e `remarcar`), e a mensagem de confirmação saiu da `Agenda`.
8	Resolvida	Os `if` aninhados viraram cláusulas de guarda: cada validador lança a exceção logo no início, e `adicionar` ficou linear (criar o agendamento, verificar a disponibilidade e guardar).
9	Resolvida	O TODO, a função `desmarcar` comentada, o cabeçalho com histórico de alterações (que já fica no Git) e os comentários de ruído foram removidos.
10	Resolvida	O comentário “lista os atendimentos do mês” foi substituído pela docstring de `listar_por_data`, que descreve o filtro por data.

3. Decisões de projeto

Decisão tomada	Alternativa descartada	Por que decidimos assim
O `Agendamento` se valida no construtor e em `atualizar_data` e `atualizar_hora_atendimento`.	Validar só dentro da `Agenda` e tratar o `Agendamento` como interno.	Toda classe pública é uma porta de entrada: com a validação só na `Agenda`, qualquer código conseguiria criar um agendamento inválido diretamente. Assim, é impossível existir um agendamento inválido.
Uma exceção própria por regra, com duas para a duração (intervalo e múltiplo).	Lançar `ValueError` genérico ou continuar com `print`.	O tipo da exceção já diz qual regra foi violada, e quem chama pode tratar cada caso separadamente. Quando a duração viola as duas regras, o intervalo é verificado primeiro.
Validar a data por ida e volta: `strptime` seguido de `strftime`, comparando com a entrada.	Usar só `strptime`, só `date.fromisoformat` ou uma expressão regular.	`strptime` sozinho aceita `2026-1-5`, e `fromisoformat` aceita `20261005`. A ida e volta recusa os dois com uma ferramenta só, sem os cuidados que uma regex exigiria.
O `ValueError` do `strptime` sobe sem conversão.	Capturar e relançar como `FormatoDataInvalidaErro`.	Segue a convenção do Python (`ValueError` para valor inválido, `TypeError` para tipo errado) e não esconde a mensagem original, que diz o que está errado (ex.: dia fora do mês). Em troca, uma data inválida pode gerar duas exceções diferentes.

4. Mudanças de comportamento

Entrada ou situação	O que acontecia antes	O que acontece agora
Marcar antes das 8h ou a partir das 18h (ex.: 20h)	Nada: os `if` falhavam e a função saía sem marcar e sem avisar	Lança `HorarioForaIntervaloPermitidoErro`
Remarcar com a hora atual ou a nova fora do expediente	Remarcava normalmente, pois `rem` não validava a hora	Lança `HorarioForaIntervaloPermitidoErro`
Duração menor que 15 ou maior que 120 minutos (inclusive zero ou negativa)	Até zero, nada era marcado e nada era avisado; de 1 a 14 e acima de 120, era aceita	Lança `ConsultaIntervaloDuracaoErro`
Duração que não é múltiplo de 15 (ex.: 20, 50)	Aceita	Lança `ConsultaMultiploDuracaoErro`
Profissional já ocupado na data e hora, ao marcar ou remarcar	Imprimia `Horario ocupado`	Lança `HorarioReservadoErro`
Remarcar um atendimento que não existe	Nada: o laço não encontrava e a função saía em silêncio	Lança `AgendamentoNaoEncontradoErro`
Data fora do padrão (ex.: `2026-1-5`) ao marcar, remarcar ou listar	Aceita como qualquer texto	Lança `FormatoDataInvalidaErro`
Data não reconhecida ou inexistente (ex.: `05/10/2026`, `2026-02-30`)	Aceita	Lança `ValueError`
Data que não é texto (ex.: `None`)	Aceita	Lança `TypeError`
Erro inesperado dentro da marcação	Engolido pelo `except: pass`	Sobe para quem chamou

5. Como executar e gerar a documentação

Item	Resposta
Comando para compilar e executar	
Comando para gerar a documentação de API	
Onde a documentação gerada ficou no repositório	
Saída do programa de demonstração	(ver abaixo)

Saída do programa de demonstração:

```
(cole aqui a saída do programa)
```

6. Uso de ferramentas de IA

Pergunta	**Resposta**
Foram usadas ferramentas de IA neste trabalho?	☐ Não ☐ Sim
Em quais etapas, e para quê?	